Klasser för samlingar av objekt

Listor, kö, stack

Iterator

Collections i Java

Abstrakta datatyper, ADT

- En abstrakt datatyp, ADT, är en datatyp som, förutom att definiera vilken sorts data som lagras, även definierar de operationer som är tillåtna på data av detta slag …
- … dvs en beskrivning av en klass i objektorienterad programmering
- Begreppet ADT används ofta när man beskriver klasser för att lagra samlingar av objekt, med specifika egenskaper som
  - Olika typer av listor
  - Stack (LIFO), Kö (FIFO)
  - Map (key – value), t.ex. ett lexikon
  - Träd
  - …

Abstrakta datatyper, ADT

- Javas API innehåller många användbara ”samlingsklasser”, som …
  - ArrayList, LinkedList, Vector
  - Queue, Stack
  - HashMap, TreeMap
  - …
- Kunskap om hur man implementerar samlingsklasser ger bra förståelse för objektorienterade begrepp som kontrakt, inkapsling, hög kohesion, gränssnitt, låg koppling m.m.
- I denna kurs ska du kunna implementera lista, stack, kö – implementerade med en intern array
  - När du skriver applikationer använder du naturligtvis klasser från Javas API

Exempel: Implementation av Stack

- Vilken typ av element som lagras anges när listobjektet skapas (generisk)
- LIFO, last in first out – element läggs till på, och tas bort från, stackens topp
  - Element ska ej kunna läggas till/tas bort från godtycklig position
- Intern privat array
  - Kan ej skapa array av okänd typ => Object[]
- Metoder:
  - push - lägg till på toppen

```
public class Stack<E> {

    private E[] elements;
    private int n;

    public Stack(int initialCapacity) {
        if (initialCapacity <= 0)
            throw new IllegalArgumentException(…);
        this.elements = (E[]) new Object[initialCapacity];
        this.n = 0;
    }
    ...

    public void push(E obj) {
        // push, and pop, objects to the right
        if (n >= elements.length) ensureCapacity();
        elements[n] = obj;
        n++;
    }
    ...

    private void ensureCapacity() {
         Object[] newArray = new Object[elements.length * 2];
         System.arraycopy(elements, 0, newArray, 0, n);
         elements = (E[]) newArray; // former array -> gc
    }
```

Exempel: Implementation av Stack

- Metoder:
  - push - lägg till på toppen
  - pop - ta bort från toppen
  - peek – titta på elementet på toppen, utan att ta bort
- NB! Inga andra metoder – add(index), remove(index) skulle bryta kontraktet

```
public class Stack<E> {

    private E[] elements;
    private int n;
    …

    public E pop() {
        if (isEmpty()) throw new IllegalStateException("empty");
        // pop, and push, objects to the right
        E toRemove = elements[n - 1];
        elements[n - 1] = null;
        n--;
        return toRemove;
    }

    public E peek() {
        if (isEmpty()) throw new IllegalStateException("empty");
        // pop, and push, objects to the right
        return elements[n - 1];
    }

    …
}
```

Exempel: Implementation av Kö

- FIFO: När ett element ska tas bort ur kön ska det element som legat längst i kön tas bort
  - Lägg till element i ena änden, ta bort från den andra änden
- Metoder:
  - enqueue – lägg till element i kön
  - dequeue – ta bort det element som legat längst i kön
  - peek – titta på elementet på nästa element att ta bort, utan att ta bort det
- NB! Inga andra metoder – add(index), remove(index) skulle bryta kontraktet

```
public class Queue<E> {

    private E[] elements;
    private int n;
    …

    // Implementation left as an exercise ☺

    …
}
```

Enkel klass för en lista

- Kontrakt för listan (enkel version)
  - Listan ska kunna lagra obegränsat antal element och utökas
  - Inga tomma positioner när element tas bort
  - Vilken typ av element som lagras anges när listobjektet skapas (generisk)
  - Operationer:
    size, isEmpty, get, add (två versioner), remove (två versioner)
- Notera att kontraktet ovan inte beskriver hur elementen i listan lagras eller hur de hanteras internt
  - Implementationen av klassen kan göras på olika sätt internt, men med samma funktionalitet dvs samma gränssnitt …
  - … lämpligt att definiera ett interface för funktionaliteten

Ett förenklat interface för listklasser

- Vi kan ha flera implementationer av list-funktionaliteten …
- … det är då lämpligt att deklarera denna funktionalitet i ett interface
- ”javadoc”-kommentarer, /** … */ beskriver vad metoden gör, kontrakt, argument etc
- isEmpty - ett exempel på en default-implementation

```
public interface IList<E> {

    int size();

    /**
     * Return true if this list contains no elements.
     * Default implementation, using the size method.
     */
    default boolean isEmpty() {
        return size() == 0;
    }

    /**
     * Returns the element at the specified position in this list.
     *
     * @param index, index of the element to return
     * @throws IndexOutOfBoundsException - if index is out of range
     */
    E get(int index);

    void add(E element);

    void add(int index, E element);

    E remove(int index);

    boolean remove(E element);
}
```

Implementation med intern array

- Intern privat array - arrayens längd >= antal element (n)
- Kan ej skapa array av okänd, generisk, typ – internt Object[]
- Arrayen utökas vid behov – hjälpmetod, ensureCapacity
- Om element tas bort flyttas element efter positionen mot denna position – hjälpmetoden pack(index)

Implementation med intern array

- Privat array, arrayens längd >= antal element (n)
- Konstruktorn skapar en array av typen Object[] – downcast till E[]
- Get m.fl. metoder: kontrollera att index är korrekt – kasta exception i annat fall - metoden checkIndex

```
public class AList<E> implements IList<E> {

    private E[] elements;
    private int n; // number of objects in elements

    public AList(int initialCapacity) {
        if (initialCapacity <= 0)
            throw new IllegalArgumentException(initialCapacity);
        this.elements = (E[]) new Object[initialCapacity];
        this.n = 0;
    }

    public AList() {
        this(10);
    }

    public int size() {
        return n;
    }

    public boolean isEmpty() {
        return n == 0;
    }

    public E get(int index) {
        checkIndex(index);
        return elements[index];
    }
```

Implementation med intern array

- Arrayen utökas vid behov – hjälpmetod, ensureCapacity

```
public class AList<E> implements IList<E> {
    …

    public void add(E obj) {
        if (n >= elements.length) ensureCapacity();
        elements[n] = obj;
        n++;
    }

    public void add(E obj, int index) {
        checkIndex(index);
        if (n >= elements.length) ensureCapacity();
        // move elements from index forward, starting from the end
        for (int i = n; i > index; i--) {
            elements[i] = elements[i - 1];
        }
        // then add obj
        elements[index] = obj;
        n++;
    }

    private void ensureCapacity() {
        Object[] newArray = new Object[elements.length * 2];
        System.arraycopy(elements, 0, newArray, 0, n);
        elements = (E[]) newArray; // former array -> gc
    }
```

Implementation med intern array

- När ett element tas bort ”packas” arrayen – elementen efter det borttagna flyttas ett steg mot början av arrayen
- remove(E obj) – söker efter obj och tar bort detta om det finns i listan
  - NB! Jämför med equals (inte ”==”)

```
public class AList<E> implements IList<E> {
    …

    public E remove(int index) {
        checkIndex(index);
        E toRemove = elements[index];
        pack(index);
        return toRemove;
    }

    public boolean remove(E obj) {
        for (int i = 0; i < n; i++) {
            if (obj.equals(elements[i])) {
                pack(i);
                return true; // removed
            }
        }
        return false;
    }

    private void pack(int fromIndex) {
        for (int i = fromIndex; i < n - 1; i++) { // avoid out of bounds
            elements[i] = elements[i + 1];
        }
        elements[n - 1] = null; // release reference
        n--;
    }
```

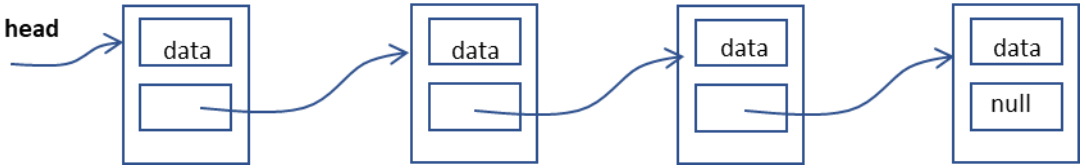
Olika typer av listor

Lista med intern array

- Internt en array med referenser
- Effektiv vid iterering över listan och vid indexering
- Ineffektiv vid insättning/borttagande av element på godtycklig position - interna arrayen måste utökas eller packas

Länkad lista

- En kedja av noder, där varje nod har data samt en länk till nästa nod
  - Noderna kan vara ”utspridda” i minnet



- Ineffektiv när vi vill komma åt ett specifikt element i listan, get(index) – vi måste ”stega” fram till elementet
- Effektiv vid insättning/borttagande av element på godtycklig position

Implementation av en länkad lista –

[NB: Du behöver förstå principen, men ej implementationsdetaljer]

- En länkad lista behöver en klass, Node, som håller data samt en referens till nästa node
- Klassen är endast för internt bruk i Listklassen – skrivs som en privat inre klass
- Listklassen har
  - En referens, head, till första noden och …
  - … en referens till sista noden, tail
- Konstruktorn allokerar inga objekt(!)

```
public class LList<E> implements IList<E> {

    private static class Node<E> {
        E data;
        Node<E> next;

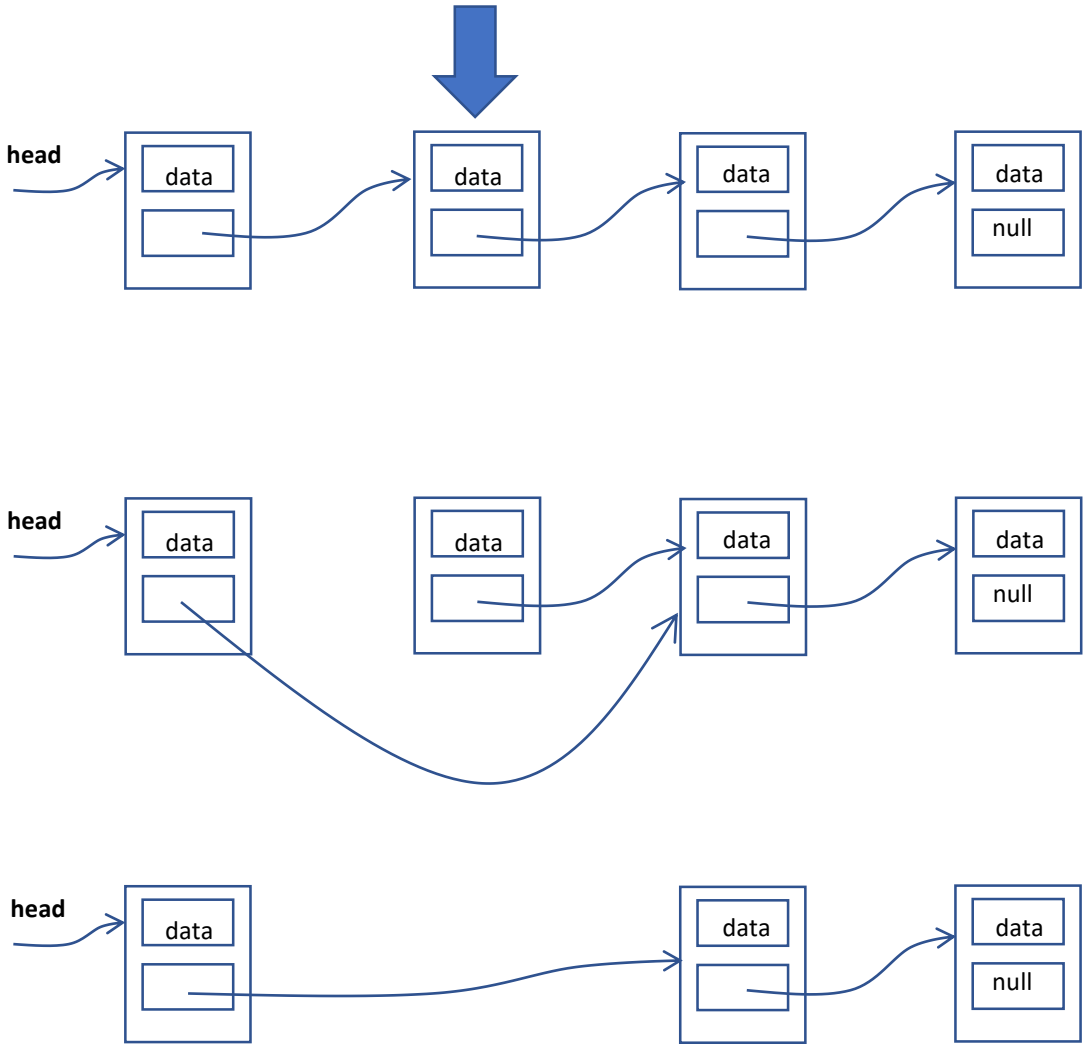
        Node(E data) {
            this.data = data;
            this.next = null;
        }
    }

    private Node<E> head;
    private Node<E> tail;
    private int n;

    public LList() {
        head = null;
        tail = null;
        n = 0;
    }
```

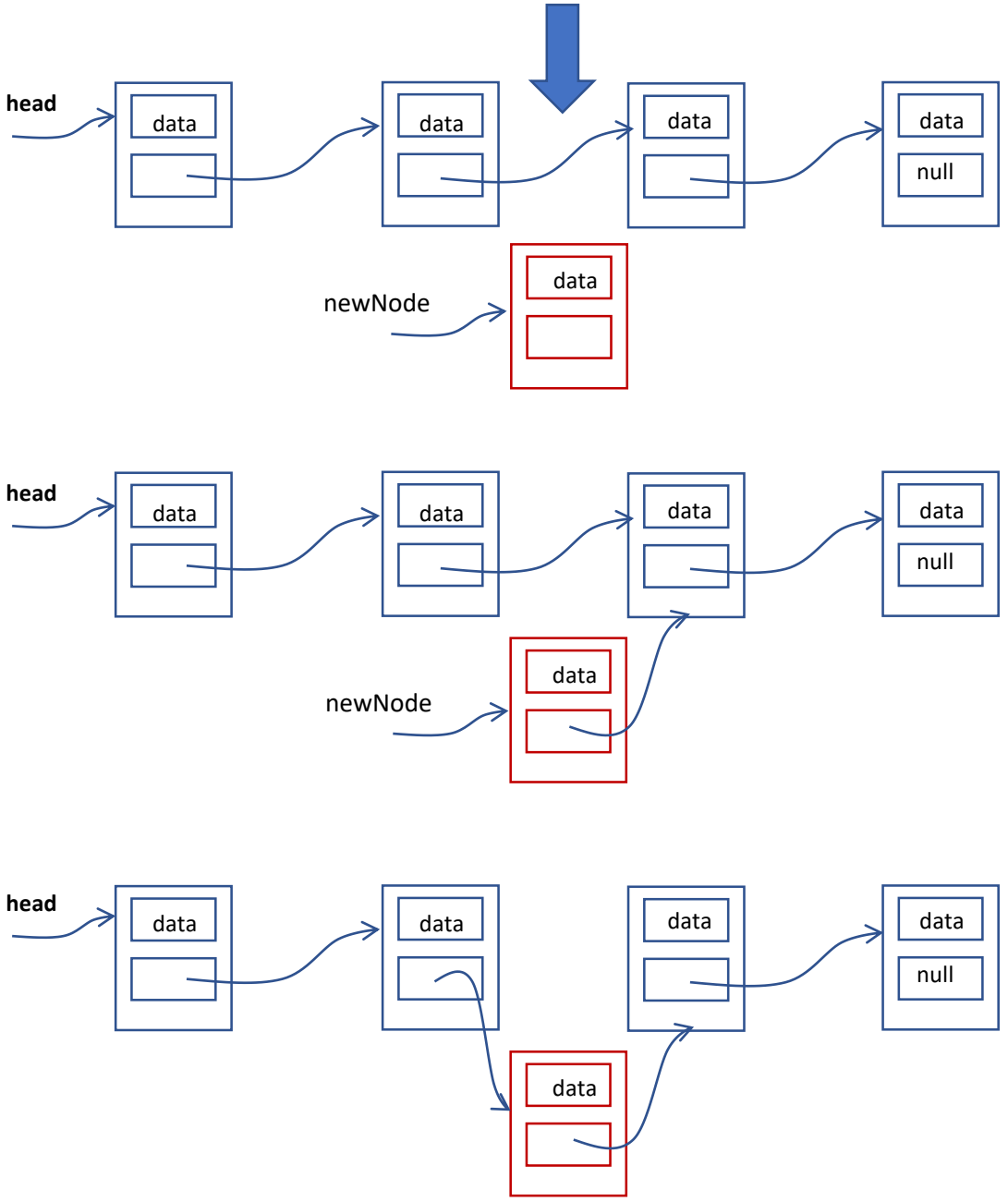
Ta bort ett element ur en länkad lista

- 1. Låt referensen next i noden före den nod som ska tas bort referera noden efter den som ska tas bort
- 2. Klart! Garbage collector tar hand om den nod som inte längre refereras
- Naturligtvis finns specialfall – ta bort första respektive ta bort sista noden



Lägga till ett element till en länkad lista

- 1. Skapa en ny nod, refererad med lokala referensen newNode
- 2. Låt next i den nya noden referera till noden efter positionen där den nya noden ska in i listan
- 3. Låt next i noden före referera till den nya noden
- Vad skulle ske hända om vi utför steg 3 före steg 2?
- Naturligtvis finns specialfall – lägga till före första respektive efter sista noden



Länkad lista – get, add (add last)

- För att hitta ett element med ett givet index måste vi stega oss fram i listan via referensen next - ineffektivt
- För att lägga ett element sist i listan använder vi referensen tail
  - Specialfall: listan är tom, dvs. head och tail är null

```
public class LList<E> implements IList<E> {
    …
    @Override
    public E get(int index) {
        checkIndex(index);
        Node<E> current = head;
        for (int i = 0; i < index; i++) {
            current = current.next;
        }
        return current.data;
    }

    @Override
    public void add(E element) {
        Node<E> newNode = new Node<>(element);
        if (tail == null) { // list is empty
            head = tail = newNode;
        } else {
            tail.next = newNode; // previous last node
            tail = tail.next;
        }
        n++;
    }
```

Ett interface för listklasser

- Om både AList och LList implementerar interfacet, IList, kan vi nu skriva kod som inte ”vet” vilken implementation av interfacet som används
- Metoder kan ha argument, och returnera, referenser av interfacetyp …
- … dvs. vilken typ av objekt som refereras är inte känt i metoden
- Ett exempel på låg koppling mellan olika delar av koden
  - ”Programmera mot interface”

```
static void testListImplementation(IList<String> list) {
        list.add("A");
        list.add("B");
        …
        System.out.println(list);
        list.add(0, "X");
        System.out.println(list);

        list.remove(0);
        System.out.println(list);
        list.remove("A");
        System.out.println(list);
}

public static void main(String[] args) {

    IList<String> lList = new LList<>();
    …
    testListImplementation(lList);

    IList<String> aList = new AList<>();
    …
    testListImplementation(aList);
}
```

Att iterera över en lista - Iterator

- Med en iterator kan vi iterera över objekten i en samling …
- … på ett av samlingen oberoende sätt
  - Fungerar på samma sätt för arrayer, för olika typer av listor, …
- Iterator är ett designmönster
- Javas ”for-each-loop” är ”syntaktiskt socker” för en iterator-loop
  - Det är alltså en iterator som används ”bakom kulisserna”

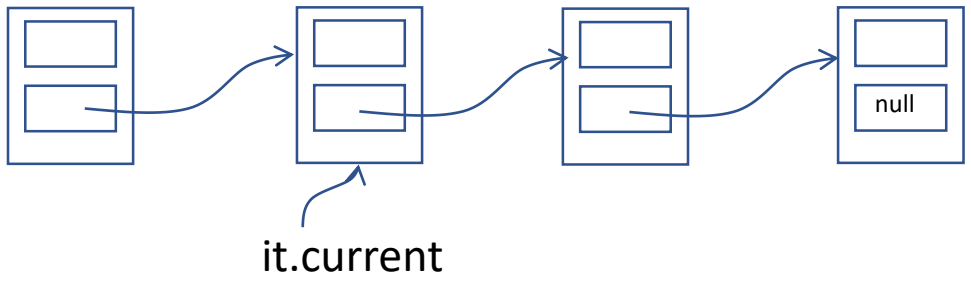
```
IList<String> list =
      new AList<String>()
      // or: new LList<String>();

Iterator<String> it = list.iterator();
while(it.hasNext()) {
      String s = it.next();
      System.out.println(s);
}

for(String s : list) {
      System.out.println(s);
}
```

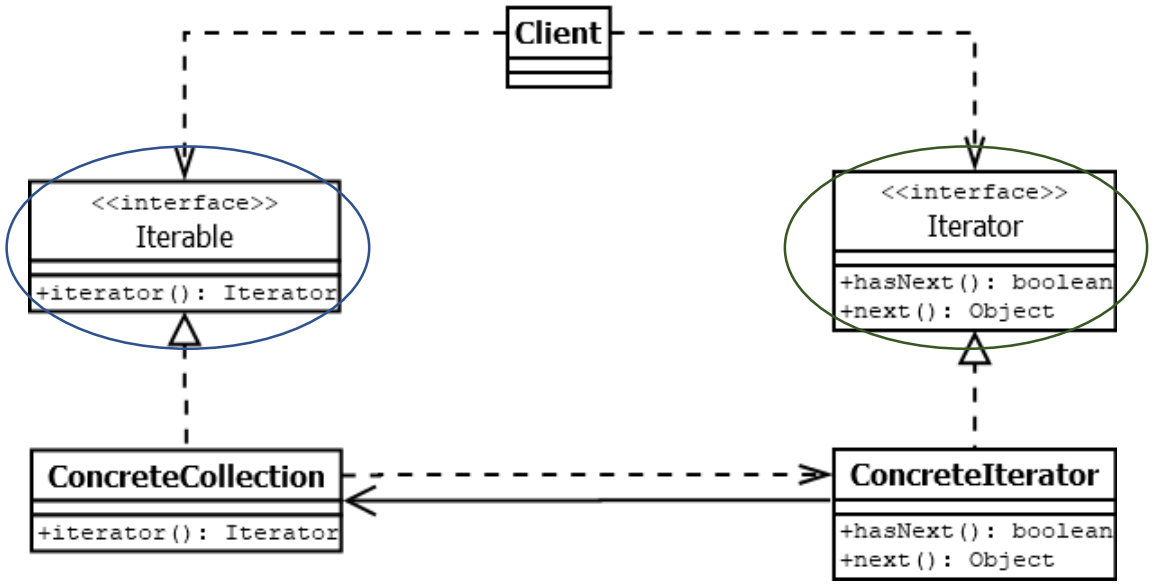
Iterator internt

- Inkapslat i iteratorn:
- Lista med intern array – en referens till den interna arrayen samt index för aktuellt element
- Länkad lista – en referens till aktuellt element som succesivt flyttas fram



Designmönstret Iterator i Java

- Listklassen ska implementera interfacet java.lang.Iterable
  - Metoden iterator() ska returnera en iterator specifik för list-objektet
- Iteratorn implementerar interfacet java.util.Iterator
  - hasNext() – ska returnera true om det finns fler element
  - next() – ska returnera nästa element
  - remove() – ska ta bort det element som senast returnerades av next() ur listan



Iterator i LList

- Klassen LList implementerar Iterable, och metoden iterator()
  - Kan göras via utökat IList-interface
- Iteratorn implementeras som en privat inre klass …
- … med en referens, current, till aktuell nod

```
public class LList<E> implements IList<E>, Iterable<E> {
    @Override
    public Iterator<E> iterator() {
        return new LListIterator<E>(head); // start at beginning
    }

    private class LListIterator<E> implements java.util.Iterator<E> {

        private Node<E> current;

        public LListIterator(Node<E> firstNode) {
            this.current = firstNode;
        }

        @Override
        public boolean hasNext() {
            return (current != null);
        }

        @Override
        public E next() {
            if (!hasNext()) throw new IllegalStateException("… null");
            E data = current.data;
            current = current.next;
            return data;
        }
    }
    …
```

Iterator i Java

- Nu kan vi iterera över en LList med hasNext och next …
- … eller använda en for-each-loop

```
LList<String> list = new LList<>();
list.add("A");
list.add("B");
list.add("C");
list.add("D");

Iterator<String> it = list.iterator();
while(it.hasNext()){
    System.out.println(it.next());
}

// for each-loop
for(String s : list) {
    System.out.println(s);
}
```

Iterator i AList

- Klassen AList implementerar Iterable, och metoden iterator()
- Iteratorn implementeras som en privat inre klass …
- … med ett index, current, till aktuellt element
- Lämnas som en övning

  Ledtråd: Ett objekt av en (icke statisk) inre klass har access till det yttre objektets data (n)

```
public class AList<E> implements IList<E>,
        java.lang.Iterable<E> {
    …
    @Override
    public java.util.Iterator<E> iterator() {
        return new AListIterator<E>();
    }

    private class AListIterator<E> implements
           java.util.Iterator<E> {

        private int current;

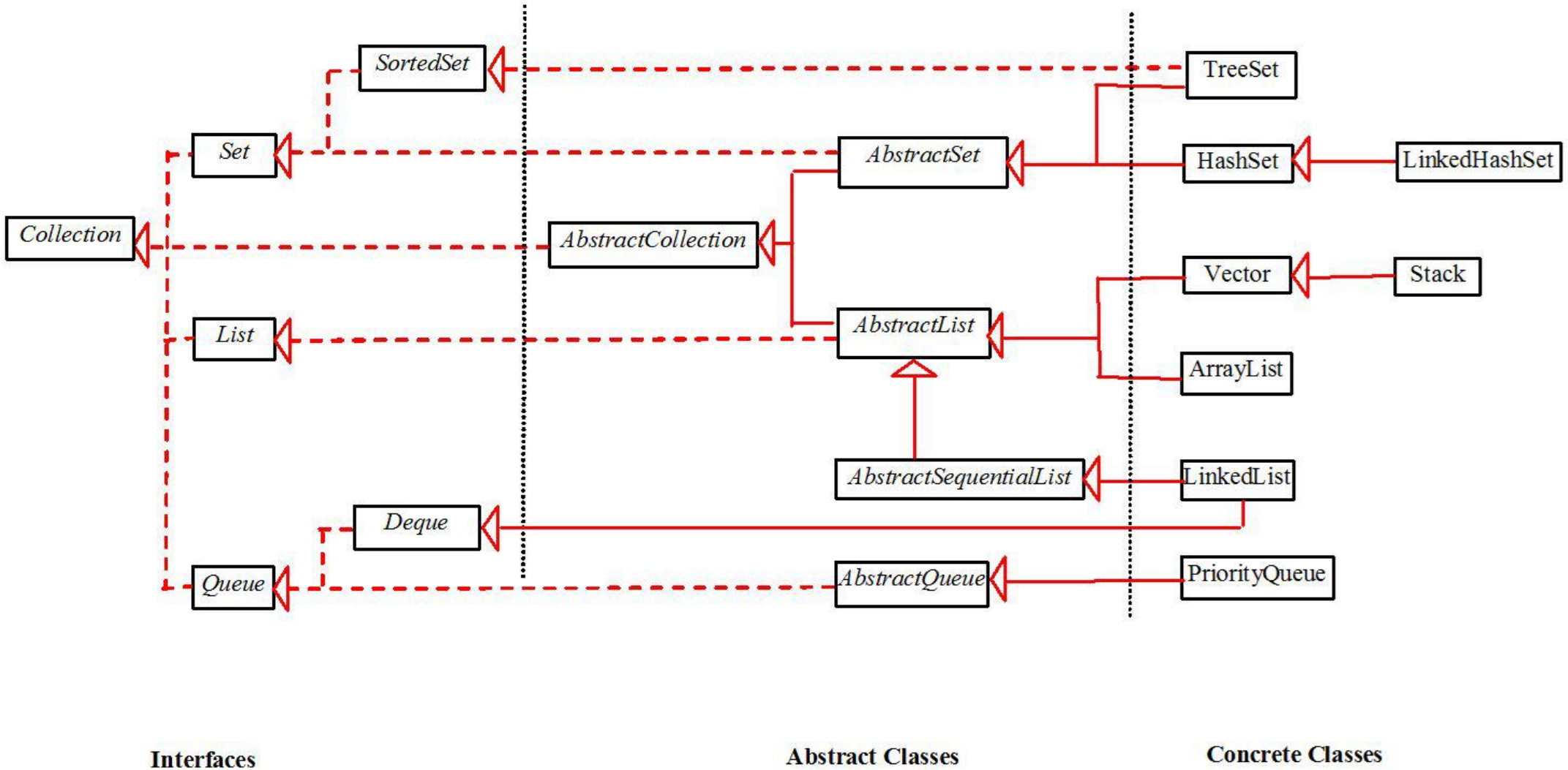
        public AListIterator() {
            // left as an exercise ☺
        }

        // left as an exercise ☺
        // …
}
```

Collections Framework i paketet java.util

- Listor, kö, map, träd, …
- Naturligtvis använder du klasser från Collections Framework när de passar ditt behov …
- … men du behöver ha förståelse, i stora drag, för hur de är implementerade för att kunna välja rätt implementation utifrån applikationens krav, som kontrakt, prestanda etc
- Hjälpklasser, som Collections, med klassmetoder som
  - sort
  - shuffle
  - stream
  - m.fl.

Collections framework i java.util



Listor och iteratorer i ”collections framework”,

- En uppsättning interface som definierar funktionalitet – List utökar Collection som utökar Iterable
- Abstrakt klass, AbstractList
  - Data och metoder som kan vara gemensamma för subklasser
- Designmönster
  - Iterator
  - Iterable, ett exempel på en ”factory method” - iteratorn är en privat inre klass och vi får endast tillgång till ett iterator-objekt via metoden iterator()

